PlayD: Crowd-Powered DJ Requests (with Smart Prioritization)

Project Description

PlayD is a two-sided application that lets partygoers request songs for a DJ through a Spotify-like browsing and search interface. Guests can submit free requests or pay to boost priority (e.g., $5 request with a $1 platform fee), which ranks the song higher in the DJ's queue so it's more visible and more likely to be played. Each request includes track metadata DJs use (*BPM, key, track/artist*) plus *context* that helps determine who requested it, how long it's been waiting, and an optional short message.

On the DJ side, PlayD provides a real-time dashboard to view and manage the request queue, filter free vs. paid requests, sort by priority and waiting time, and mark outcomes (played/skipped/saved). PlayD monetizes by taking a fee from paid boosts while still allowing free requests to keep the experience inclusive and high engagement.

Problem Statement

At live events, song requests are chaotic (people yelling, texting, interrupting the booth) and rarely include the information DJs need to act fast: an exact track match, mix-friendly metadata (BPM/key), and the social context behind why a song matters. DJs also don't have a structured way to triage requests by priority (especially when money is involved) while keeping the vibe fair and maintaining control of the set. This friction leads to guest frustration ("they ignored my request"), DJ disruption ("I'm constantly interrupted"), and missed revenue opportunities for DJs and venues. PlayD solves this by turning requests into a clear, ranked, metadata-rich queue with optional paid prioritization and guest context (requester + message + time-waiting) to support better, faster decisions.

OKRs (Objectives & Key Results)

Objective 1: Make requesting fast, accurate, and contextual for guests

- KR1: Median time from search to successful request submission ≤ **30 seconds**.
- KR2: ≥ **90%** of requests attach to a valid track entity (minimizes typos/ambiguity).
- KR3: ≥ **95%** of requests include BPM + key on the DJ view (via integrated metadata or fallback logic)

Objective 2: Give DJs a clean workflow to triage and act on requests in real time

- KR1: DJ can filter between **Free vs. Paid** and see priority-ranked queue in ≤ **2 interactions** from landing
- KR2: New requests appear on the DJ dashboard with median latency ≤ **2 seconds**.
- KR3: Each queue item displays: **track/artist, BPM, key, requester, time since first requested, and message** with ≥ **99%** field render success (when data exists).

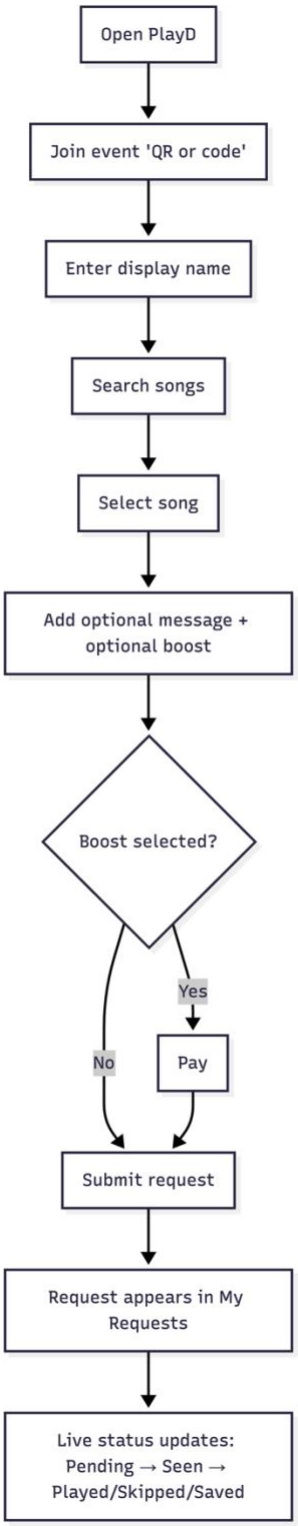
Objective 3: Validate monetization that is transparent and doesn't harm DJ control

- KR1: Payment success rate ≥ **98%** for paid boosts (excluding user abandonment).
- KR2: 100% of paid transactions log **gross amount, platform fee, and DJ payout** in a ledger.

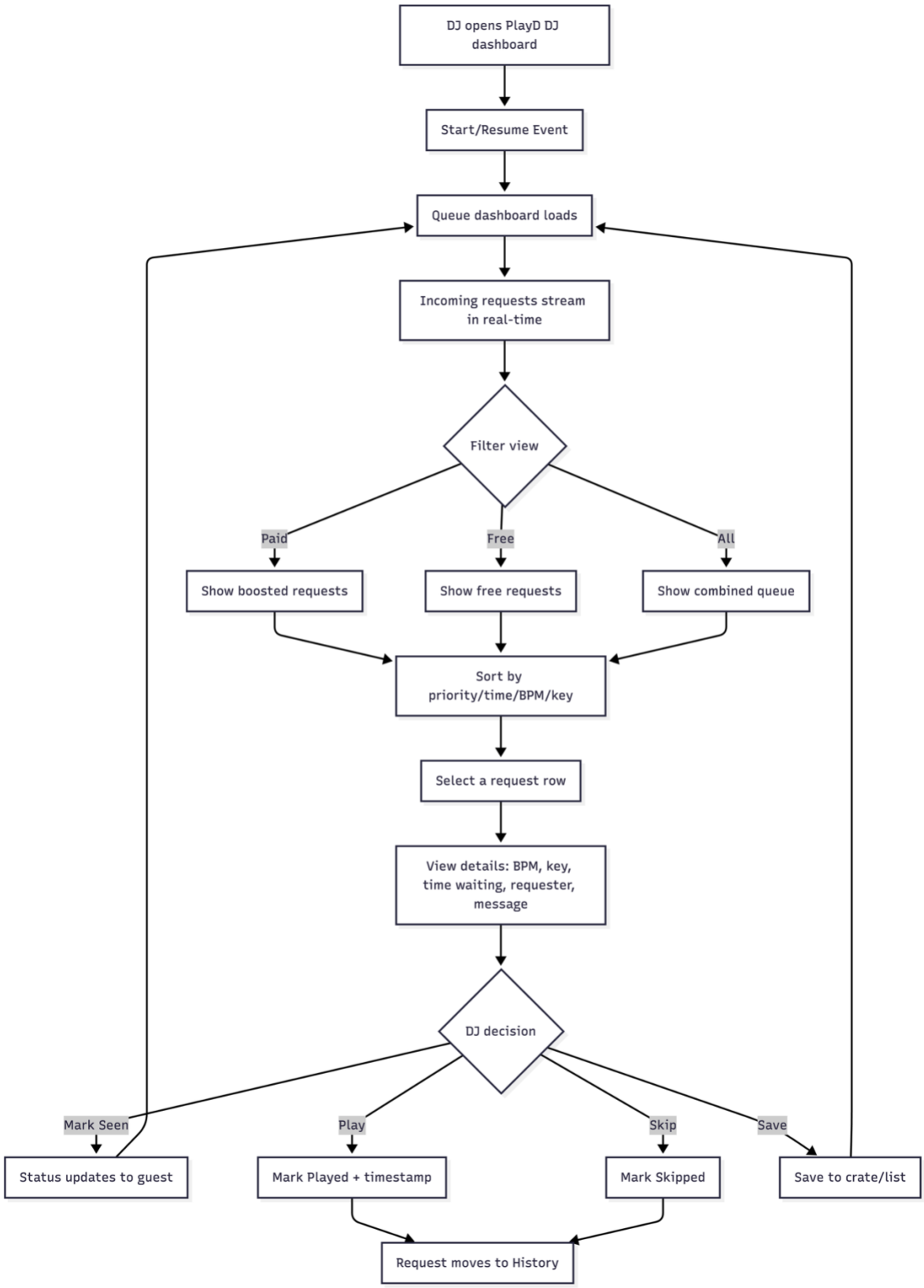
Solution discovery

- **User research:** 5–8 quick interviews (3–4 DJs, 2–4 frequent partygoers) to confirm pain points, must-have metadata (BPM/key), and what "fair" paid priority looks like.
- **Prototype testing:** low-fi Figma of both dashboards; run task tests (submit request, DJ triage) and measure time-to-complete + confusion.
- **Pilot events:** 2 small live trials (one house party, one campus event). Track metrics: requests/min, % duplicates, median wait time, % played, and paid boost conversion.

Workflow 1: Guest request flow



Workflow 2: DJ Interaction flow



Technical breakdown of major features

MVP Phase (1 – 3 Weeks)

Goal: working guest request → DJ queue loop for one live event.

1. **Event Join (QR / Event Code) — S (3–5 days)**

- **Req:** DJ starts event → code + QR. Guests join event via code/QR.
- **Verification:** valid join routes correctly; invalid code errors; DJ can end event.

2. **Guest Search + Request (single track) — M (1–1.5 weeks)**

- **Req:** Spotify-like search → select track → submit request (+ optional message).
- **Verification:** request reaches DJ view quickly; guest sees pending status.

3. **DJ Queue (core list + status) — M (1–1.5 weeks)**

- **Req:** DJ sees queue with: track/artist, **BPM**, **key**, requester, time-waiting, message icon; can mark Seen/Played/Skipped.
- **Verification:** status updates reflect on guest side.

4. **BPM + Key Enrichment (best-effort) — S–M (3–7 days)**

- **Req:** store BPM/key when available; show "—" if missing.
- **Verification:** DJ view displays metadata for most tracks.

5. **DJ Login (basic auth) — S (2–4 days)**

- **Req:** only DJs can access DJ dashboard/start events.
- **Verification:** protected routes enforced.

Main Build Phase (1 - 4 weeks)

Goal: monetization + scalable queue management.

6. **Payments + Boosted Priority + Platform Fee Ledger — L (2–3 weeks)**

- **Req:** guest pays tiered boost; platform fee recorded; DJ sees boosted ranking.
- **Verification:** fee breakdown shown pre-pay; paid labeled; ledger totals reconcile.

7. **DJ Filters/Sorting — M (1 week)**

- **Req:** filter All/Paid/Free; sort by priority/time/BPM/key/recency.
- **Verification:** stable sorting; fast UI.

8. **Message + Mic Note Controls — M (1 week)**

- **Req:** optional guest message + mic-allowed flag; DJ can disable mic notes globally.
- **Verification:** message visible; mic tag honored by DJ settings.

9. **Real-time Updates (queue + statuses) — M (1–1.5 weeks)**

- **Req:** DJ queue updates live; guest status updates live.
- **Verification:** near-real-time propagation.

10. **DJ Event Controls — S–M (4–7 days)**

- **Req:** pause requests; toggle free/paid; set tiers; end event.
- **Verification:** paused/ended states enforced for guests.

Stretch Goals Phase (1 – 2 Weeks)

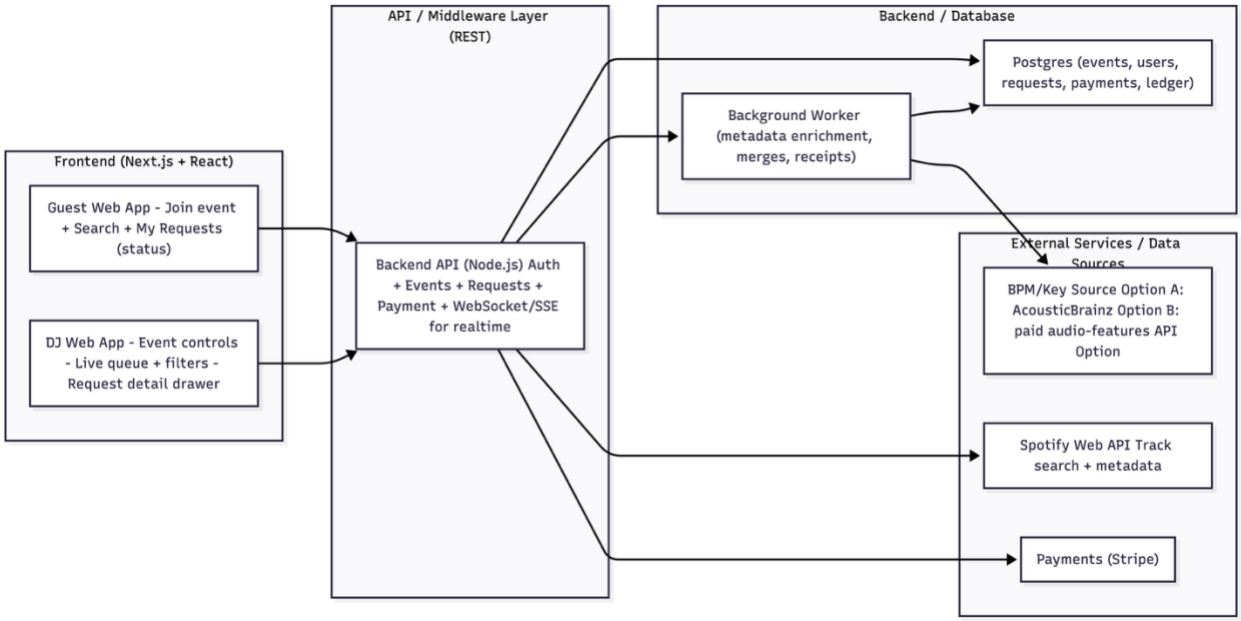
11. **Export / DJ Software-Friendly Output (CSV) — M (1 week)**

- **Req:** export queue with metadata + messages/requesters.
- **AC:** correct file + columns.

12. **Analytics + Post-event Summary — M–L (1–2 weeks)**

- **Req:** stats: requests, boosts, wait time, played rate, top tracks.
- **AC:** per-event dashboard + export.

Architecture:



Technologies

- **Frontend:** Next.js (React) for Guest + DJ dashboards; WebSockets client for live updates
- **API: REST** (plus WebSockets for realtime) via **Node.js** (NestJS)
- **Data: Postgres** (core system of record); background **worker** for async tasks
- **Payments:** Stripe

Data sources

- **Spotify Web API** for search + track metadata.
- **BPM/Key:** plan a separate source because Spotify restricted access to Audio Features/Analysis for new Web API use cases.
    - Option: **AcousticBrainz** (open)
    - Option: paid audio-features provider (higher reliability)

Tradeoffs (brief)

- **REST vs GraphQL:** REST is simpler for queue/payments; GraphQL adds schema overhead.

- **Postgres vs Mongo:** Postgres fits payments integrity + relational queries; Mongo is flexible but harder for strict accounting.
- **Node vs FastAPI:** Node often smoother for realtime; FastAPI is fast to build typed REST

Design approaches

Layered architecture

- Controllers (REST) → Services (business logic) → Repositories (DB)
- Keeps queue/payments logic testable and clean

Event-driven worker pattern

- Request created → enqueue "metadata enrichment" job → update request with BPM/key
- Avoids blocking the user flow on external APIs